COSC 461/561 Project 1: Eagle Delivery

Andres Iniguez (ID 02704032) · A* search and Q-learning for a simulated EMU delivery robot

Part A: Planning a delivery with A*

(a) Problem formulation. *State:* the robot's cell (row, column) on a walkable square (`.` or a building letter). *Initial state:* the start entrance's cell, e.g. S = (1, 1). *Actions:* UP, DOWN, LEFT, RIGHT. *Transition model:* deterministic; an action moves one cell if the target is walkable and otherwise leaves the robot in place (search only generates the moves that change position, via `campus.neighbors`). *Goal test:* the state equals the destination entrance's cell; passing other entrances does nothing. *Path cost:* the number of actions, each costing 1. **BFS is optimal here** because it removes states in order of depth, and with unit step costs depth equals path cost. A state's first discovery is therefore by a shortest route, and the goal is first removed at the minimum number of actions.

(b) Admissibility of Manhattan distance. One action changes either the row or the column by exactly 1 (no diagonals), so it reduces $|r_s - r_g| + |c_s - c_g|$ by at most 1. Reaching the goal therefore takes at least h(s) actions. Walls only remove moves and never create shortcuts, so the true cost $h^*(s) \geq h(s)$, and h(goal) = 0. h is also *consistent*: for any move $s \to s'$, $h(s) \leq 1 + h(s')$. So the first time A* removes a state from the queue as a non-stale entry, it already has that state's optimal cost.

(c) Implementation. A `heapq` queue holds (f, insertion number, g, state), so equal-f ties break in insertion order. `best_g` and `parents` are dictionaries. A successor is pushed again only when a strictly cheaper g is found. Popped entries whose g differs from `best_g` are skipped as stale. The goal test happens on removal. A node counts as expanded when its successors are generated, the same convention as the supplied BFS.

Table 1. BFS vs. A (results/search_comparison.csv). The first six rows are the required tasks; the last row is the custom map from Part C.*

Map	Delivery task	BFS route length	A* route length	BFS expanded	A* expanded	A* saving
emu_campus.txt	S → W	18	18	67	22	67%
emu_campus.txt	L → D	22	22	79	62	22%
emu_campus.txt	H → P	16	16	41	16	61%
emu_detour.txt	S → W	26	26	70	51	27%
emu_detour.txt	L → D	22	22	65	60	8%
emu_detour.txt	H → P	16	16	41	16	61%
my_campus.txt	S → W	20	20	83	25	70%

Discussion. Both algorithms return routes of **equal length in every row**, as expected: BFS is optimal for unit costs and A* is optimal with an admissible, consistent heuristic. A* expanded fewer nodes in all seven rows, but the saving ranges from 67–70% (campus S→W, my_campus S→W) down to **only 8%** (detour L→D, 65 vs. 60). A* is not guaranteed to expand fewer nodes. It helps least when walls force the route far from the straight line, so the true cost is much larger than h. L sits at (5, 1) and D at (5, 15) in the same row, so h = 14, but L is walled in on its right side and the route takes 22 actions. Every cell whose g + h is below 22 must be expanded, and that covers most of the map. In the same way, the detour's closure at (3, 6) lengthens S→W from 18 to 26, and A*'s expansions rise from 22 to 51 while BFS barely changes (67 to 70). When h is uninformative, A* behaves like uniform-cost search, which is BFS order here, and it also pays heap overhead on each expansion.

Part B: Learning a delivery policy with Q-learning

Implementation and settings. `epsilon_greedy` draws `rng.random()`. If the draw is below ε it picks uniformly among all four actions. Otherwise it picks uniformly among the actions tied at the maximum Q, with unseen pairs worth 0.0. `q_learning_update` sets $Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)]$, where y = r on the terminal step and $r + \gamma \max_{a'} Q(s',a')$ otherwise, and changes only that one entry. All baseline runs use the starter defaults: emu_campus, 2,000 training episodes, seeds 0/1/2, 100 evaluation episodes, 200 max actions, $\alpha = 0.3$, $\gamma = 0.95$, constant $\varepsilon = 0.2$, slip = 0. Evaluation uses $\varepsilon = 0$ with a frozen Q-table.

Table 2. Baseline results, means over seeds 0–2 (training_seed.csv, evaluation_seed*.csv; computed by analyze_results.py into results/training_analysis.csv). Truncated = hit the 200-action limit.*

Task (optimal)	Eval. success	Eval. steps / return	Train success	Trunc. eps.	Last failure (seeds 0, 1, 2)	Steps, eps. 1–100	Steps / return, last 200	Total train steps
S → W (18)	100%	18 / 22	99.2%	17	41, 33, 32	98.0	22.5 / 17.5	52,747
L → D (22)	100%	22 / 18	99.6%	8.7	17, 14, 12	93.9	27.8 / 12.2	62,328
H → P (16)	100%	16 / 24	99.2%	15.3	149, 39, 36	57.6	20.0 / 20.0	44,122

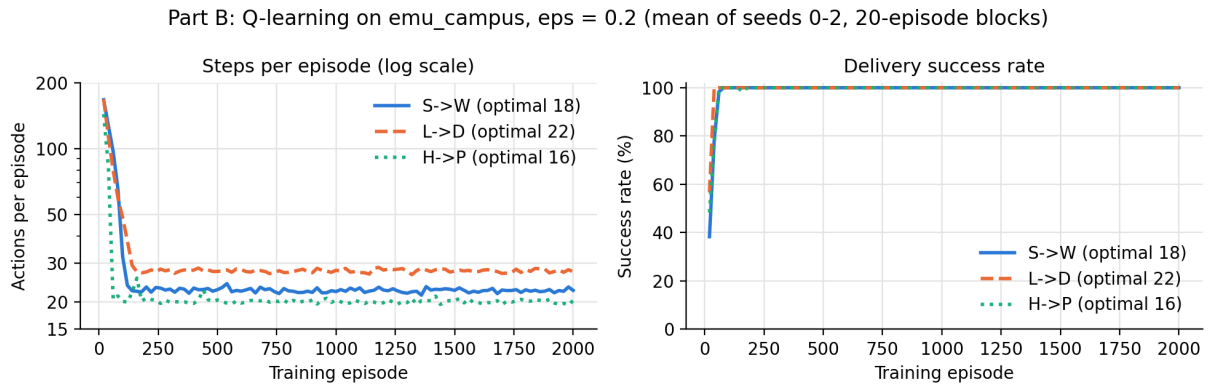


Figure 1. Training steps per episode (left, log scale) and delivery success rate (right) for the three campus tasks, averaged over seeds in 20-episode blocks.

Observations. (1) **All nine runs reach 100% evaluation success on the shortest route**: 18 actions with return 22 for S→W, 22 with return 18 for L→D, and 16 with return 24 for H→P, exactly the BFS/A* lengths. The sample SD across seeds is 0 for both success rate and return. (2) **Learning is fast.** Every truncated training episode happens early: the last failure is by episode 41 for S→W and 17 for L→D, and by episode 39 for H→P except one late truncation at episode 149 (seed 0). No run fails after episode 200, and training success is above 99%. Figure 1 shows the steps per episode falling from about 170 to a plateau within roughly 150 episodes. H→P learns fastest (57.6 mean steps in episodes 1–100 vs. 94–98) because its optimal route is the shortest (16 actions). (3) **Training stays above optimal while evaluation does not.** Over the last 200 episodes, training averages 22.5, 27.8 and 20.0 actions, 4–6 above optimal. With constant ε = 0.2, about one action in five is random. A random move often bumps a wall or steps backwards, and each one costs at least 1–2 extra actions. The longest task, L→D, makes the most random moves and pays the most. Evaluation sets ε = 0, so the greedy policy follows the learned optimal route every time. The gap between training and evaluation is the cost of exploration, not a learning failure.

Part C: Custom map and exploration experiment

(i) Question and prediction (stated before running the experiment). *How does the exploration rate ε affect learning speed, training performance and the final greedy route on my_campus S→W?* I predicted that (1) training episodes would grow longer as ε rises, because more actions are random; (2) ε = 0.05 might settle on the longer 28-action inner route because it explores so little, while ε = 0.5 would find the 20-action route; and (3) ε = 0.5 would learn the greedy policy fastest, since it visits more state-action pairs early.

(ii) The custom map. The map passes every rule (check_map.py). Each letter appears once, it is 15×19, rectangular and walled, and every open cell is reachable. It has 12 independent cycles. The relevant **loop** runs around the wall block in row 2, columns 9–15, and gives two S→W route choices. The **top corridor** takes 20 actions: row 1 is blocked at (1, 7), so the route dips through (3, 6)–(3, 8) and climbs back to row 1 at (2, 8). The **inner route** takes 28 actions: it stays on row 3 to (3, 9), drops to (5, 9)–(5, 11), returns to row 3 and follows it to column 15, then goes via (5, 15)–(5, 17) and up the east column to W. **Dead-end branches** end at (11, 16), (13, 3), (13, 5) and (13, 11), and the H and P entrances are cul-de-sacs. The whole southern half (L, P, D and the lower loops) is a detour trap for an S→W robot. BFS and A* both find the 20-action route; BFS expands 83 nodes and A* 25 (Table 1).

```
####################
#S.....#.........W#
#.###..#.#######..#
#.#.......#.....#.#
#.#.#####.#.###.#.#
#L#.#...#...#H#...#
#.#.#.#.#####.#.###
#...#.#.......#..D#
###.#.#########.#.#
#P....#.....#.....#
#####...###.#.#####
#.......#.....#..##
#.#####.#.##....###
#...#.....#..#..###
####################
```

my_campus.txt (15 × 19). S (1,1), W (1,17), L (5,1), H (5,13), D (7,17), P (9,1).

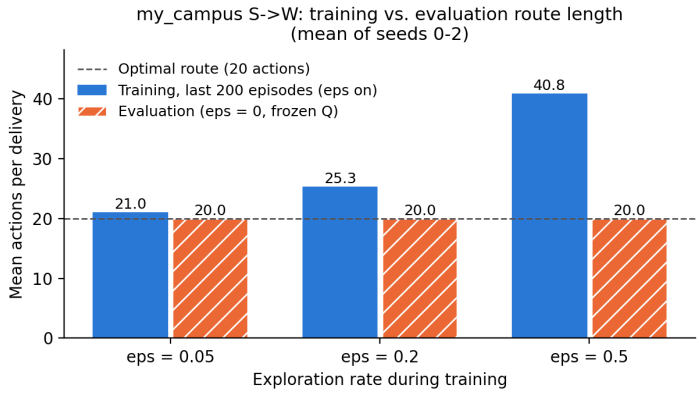


Figure 2. Late-training vs. evaluation steps by ε.

(iii) Method. *Changed:* only ε, set to 0.05, 0.2 and 0.5 (constant during training). *Held fixed:* the map, S→W, seeds 0/1/2, 2,000 training episodes, α = 0.3, γ = 0.95, 200 max actions, slip = 0 and 100 evaluation episodes. Each seed and setting starts from a fresh, empty Q-table. Training runs the supplied loop: choose an ε-greedy action, step, apply the Q-learning update, and repeat until delivery or 200 actions. Evaluation then uses ε = 0 with the Q-table frozen. This gives 9 runs, saved in results/custom_e005,

custom_e020 and custom_e050.

Table 3. ε experiment on my_campus S→W, means over seeds 0–2 unless listed per seed (optimal route = 20 actions, return 20).

ε	Eval. success	Eval. steps / return	Train success	Trunc. eps.	Trunc. after ep. 200	Last failure (seeds 0, 1, 2)	Steps, eps. 1–100	Steps / return, last 200	Total train steps
0.05	100%	20 / 20	97.5%	50.3	0, 0, 1	145, 99, 332	130.3	21.0 / 19.0	53,215
0.2	100%	20 / 20	97.4%	52.7	0, 1, 0	99, 581, 123	132.5	25.3 / 14.7	61,366
0.5	100%	20 / 20	97.0%	59.3	1, 3, 2	1329, 1652, 1157	141.4	40.8 / -0.8	93,352

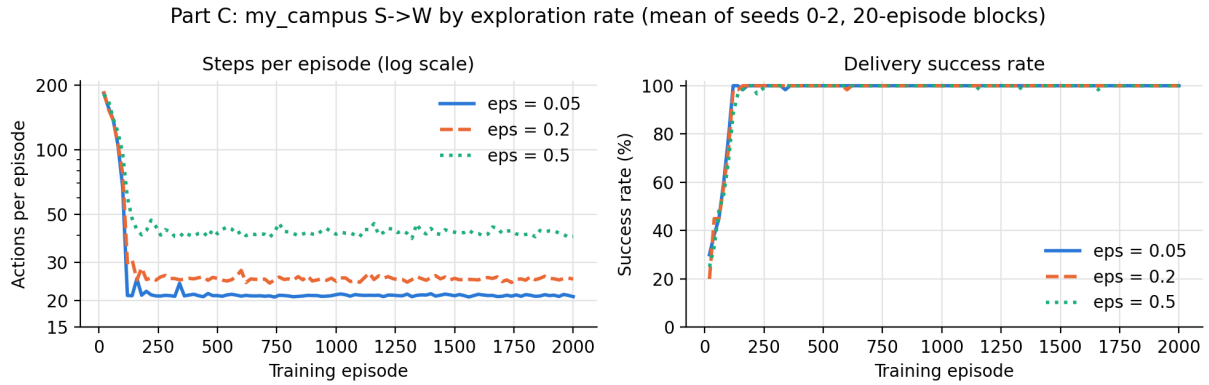


Figure 3. my_campus S→W training steps per episode (left, log scale) and success rate (right) by ε, averaged over seeds in 20-episode blocks.

(iv) Findings. Final performance did not depend on ε. All nine runs deliver 100% of evaluation episodes in 20 actions (return 20), using the top corridor. **Training cost depends strongly on ε.** Over the last 200 episodes, training averages 21.0, 25.3 and 40.8 actions for ε = 0.05, 0.2 and 0.5, with returns of 19.0, 14.7 and -0.8. ε = 0.5 needs about 75% more total training actions (93,352 vs. 53,215), and its best late episode is still 23–24 actions. **Early learning is almost identical.** Mean steps over episodes 1–100 are 130.3, 132.5 and 141.4, and every setting has about 50–60 truncated episodes, nearly all before episode ~150 (Figure 3). The difference is in late failures: ε = 0.5 still truncates 1–3 episodes per seed after episode 200, with the last at episodes 1,157–1,652. The smaller ε values have at most one late truncation per run.

(v) Discussion. Prediction (1) held. Predictions (2) and (3) did not: ε = 0.05 never settled on the 28-action route, and ε = 0.5 did not learn faster. The reason is that **Q starts at 0 and every step costs −1**, so any action already tried has a negative value, while untried actions still look like 0. The greedy choice therefore prefers untried actions. This acts like *optimistic initialization*: early on, even a nearly greedy agent explores systematically, and ε adds little. That explains the near-identical first 100 episodes and truncation counts. Q-learning is also *off-policy*. Its target uses $\max_a Q$, so random actions do not bias the learned values toward a worse route, and all settings converge to the same optimal greedy policy. In this deterministic map, ε mainly sets the *price* of exploration during training, not the quality of the final route. The surprise was that a large ε can still cause failed episodes late in training: when half the actions are random, a long random excursion into the southern loops sometimes uses up the 200-action limit even with a correct Q-table. With slip or a non-stationary map, I would expect larger ε to matter more, which would be a natural follow-up.